

T1 - Design de Software

EDUARDO FARIA KRUGER

FERNANDO GBUR DOS SANTOS

THALITA MARIA DO NASCIMENTO

Novembro 2025

Sumário

1	Requisitos Relevantes	2
2	Arquitetura Proposta	2
2.1	Visão Geral da Arquitetura	2
2.2	Componentes Principais	2
3	Diagramas	3
3.1	Diagrama de Componentes	3
3.2	Diagrama de Classes	3
4	Justificativa das Decisões Arquiteturais	4
5	Considerações finais	4

1 Requisitos Relevantes

Consistência, usabilidade, confiabilidade e escalabilidade são os requisitos arquitetonicamente significativos (ASR). A consistência visa evitar *double-booking*; a usabilidade está diretamente relacionada à simplicidade do sistema; a confiabilidade se relaciona a disponibilidade da solução para uso dos usuários e a escalabilidade está relacionada à capacidade de expandir e escalar um sistema. O maior *trade-off* é entre os ASR consistência e escalabilidade, pois não é possível de se garantir os dois ao mesmo tempo, visto que para escalar uma solução mais unidades de banco de dados são necessárias, o que gera inconsistências. Arquiteturas de microserviços e *event-driven* são boas soluções para escalabilidade, mas em razão da simplicidade da necessidade de Dona Maria, das operações a serem realizadas no sistema e da quantidade de usuários que terão o poder de realizar as reservas (apenas os funcionários de cada filial, que terão contas de administradores), escolheu-se a arquitetura visando garantir consistência, uma dor que era observada pela Dona Maria, assim como diminuir gastos.

2 Arquitetura Proposta

A arquitetura escolhida é um cliente-servidor monolítico em camadas.

2.1 Visão Geral da Arquitetura

Cada cliente acessa via navegador um servidor (único) que contém as quatro camadas do sistema:

- **Camada de apresentação (Front-end):** uma aplicação web básica acessada via browser;
- **Camada de regra de negócio (Middleware):** uma camada que impede o *double-booking*
- **Camada de lógica (Back-end):** uma API REST que recebe requisições do front-end, interage com o banco de dados e devolve as informações processadas;
- **Camada de persistência e dados:** o próprio banco de dados relacional.

2.2 Componentes Principais

- **Front-end:** Uma Single Page application (SPA) contendo os dados de contato e o CRUD das entidades principais da plataforma

- **Middleware** Camada intermediária entre front-end e back-end. É uma função *verificaDisponibilidade* que garante que não haja *double-booking* de reservas ao ser executada antes de requisições POST e PUT da entidade Reserva
- **Back-end:** Uma API NodeJS usando Express para comunicação com o banco de dados divididas em rotas, onde cada arquivo {entity}Routes.ts contém as respectivas rotas (CRUD) daquela entidade específica
- **Banco de dados:** Um bando postgresQL com schema definido e uma tabela para cada entidade

3 Diagramas

Nesta seção são apresentados os principais diagramas UML que representam a estrutura e organização do sistema desenvolvido.

3.1 Diagrama de Componentes

Visão geral da arquitetura proposta

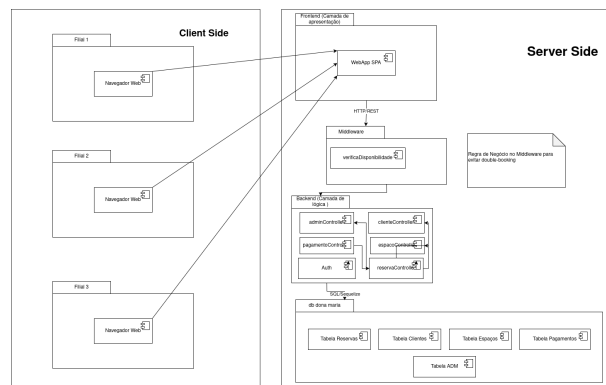


Figura 1: Diagrama de Classes do Sistema

3.2 Diagrama de Classes

Visão sobre as entidades e relacionamentos entre elas

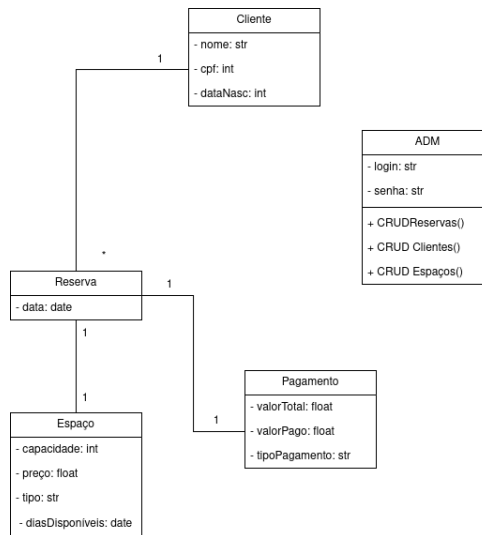


Figura 2: Diagrama de Classes do Sistema

4 Justificativa das Decisões Arquiteturais

Dada a simplicidade do problema e a baixa necessidade de investimento em infraestrutura, acreditamos que a melhor arquitetura para esse caso seja a arquitetura em camadas + cliente-servidor.

Apesar de simples, ela supota uma quantidade considerável de requisições, mais do que o suficiente relatado para o problema. Além disso, a manutenção é barata e inserir novas filiais é muito simples, já que cada filial será uma conta ou login a mais no sistema com privilégios de administrador.

5 Considerações finais

Dada a implementação feita e as experiências com as plataformas MecRed e Ademir, concluímos que essa arquitetura é estável, funciona bem para a quantidade de usuários proposta pelo problema e a gerência de filiais é muito facilitada considerando apenas os usuários administradores. Arquiteturas como micro serviços e orientada a eventos também funcionaria para resolver os problemas de escalabilidade, porém o problema proposto é simples e o gasto com outras arquiteturas não é justificado na opinião do grupo.